

Angular Learning Series

Sections 13–16 Retrofit

Topics 14 through 26

Study Plan Checkpoints · Interview Questions · Anti-Patterns · Mental Models

Generated: 16 May 2026

Topics covered in this document:

Topic 14 — readonly & Immutability Patterns

Topic 15 — Access Modifiers (private, protected, public)

Topic 16 — TypeScript Generics

Topic 17 — TypeScript Decorators

Topic 18 — Error Handling

Topic 19 — Optional Chaining & Nullish Coalescing

Topic 25 — Component Lifecycle Hooks

Topic 26 — @Input() & @Output() in Depth

TOPIC 14 — readonly & Immutability Patterns

TypeScript | Intermediate | High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 1 — let / const / var (const is the foundation of immutability intent; readonly extends that intent to object properties)
- Topic 7 — Pass by Value vs Pass by Reference (understanding why const does not protect object contents is essential before readonly makes sense)
- Topic 13 — TypeScript Interfaces & Types (readonly is applied to interface and type properties; you must be comfortable defining both before this topic lands properly)

Unlocks:

- Topic 15 — Access Modifiers (readonly is often used alongside private/public in class constructors; the two topics compose naturally)
- Topic 16 — TypeScript Generics (Readonly and DeepReadonly patterns are generic utility types; this topic gives you the motivation for them)
- Topic 25 — Component Lifecycle Hooks (immutable input patterns directly affect how OnPush change detection fires; you will reference this topic there)

Production Readiness Assessment:

After studying readonly and immutability patterns alone you can write safer interfaces and service contracts, but you will not be production-confident until you combine this with OnPush change detection (Topic 25) and @Input() patterns (Topic 26), because immutability in Angular is most consequential at the component boundary, not inside isolated classes.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "What is the difference between const and readonly in TypeScript, and when would you use each?"

What a Strong Answer Covers:

- const prevents reassignment of a variable binding; readonly prevents reassignment of an object property
- const does not make the object it points to immutable — its properties are still mutable
- readonly is applied at the property level in interfaces, type aliases, and classes
- readonly is enforced only at compile time; it disappears at runtime
- Practical rule: use const for variable declarations, readonly for object shape contracts

What a Weak Answer Misses: Weak answers treat const and readonly as interchangeable or fail to mention that const gives no protection over the object's internal properties.

Question 2 — Mid

The Question: "How does immutability relate to Angular's OnPush change detection strategy, and how would you enforce it in a component that receives complex objects as inputs?"

What a Strong Answer Covers:

- OnPush triggers change detection only when an input reference changes, not when properties inside an existing object mutate
- Mutating an object in place silently breaks OnPush components — the view never updates
- The correct pattern is to produce a new object reference on every state change (spread operator, Object.assign, or immer)
- readonly on @Input() properties signals intent but does not enforce immutability at runtime — you must also discipline the parent component
- Angular signals and the new signal-based inputs push this pattern further by making reactivity explicit

What a Weak Answer Misses: Weak answers describe OnPush in isolation without connecting it to the concrete danger of reference equality checks failing when objects are mutated in place.

Question 3 — Senior

The Question: "You are designing a shared state service used by a feature module with twelve components, several of which use OnPush. How would you architect the service to prevent accidental mutation, and what are the trade-offs of each approach?"

What a Strong Answer Covers:

- Option 1 — expose state as readonly signals or BehaviorSubjects and return frozen objects (Object.freeze) from selectors; trade-off: Object.freeze is shallow and has a runtime cost
- Option 2 — use the Readonly and ReadonlyArray utility types on the service's public API surface; trade-off: compile-time only, a developer can cast around it
- Option 3 — adopt immer or a structured immutability library; trade-off: adds a dependency and a learning curve for the team
- Option 4 — NgRx or SignalStore with selectors as the canonical immutable projection layer; trade-off: overhead justified only at scale
- The architectural decision is about where you put the enforcement burden — TypeScript types (zero runtime cost, bypassable), Object.freeze (runtime cost, reliable), or a state library (opinionated, scalable)

What a Weak Answer Misses: Weak answers propose one approach without acknowledging that all immutability enforcement in TypeScript is either compile-time only or carries runtime cost, and that the right choice depends on team discipline and codebase scale.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Confusing const with Immutability

What it looks like:

```
// Developer believes this object is immutable
const config = { apiUrl: 'https://api.example.com', timeout: 3000 };

// Later in a service — no TypeScript error, no runtime error
config.timeout = 99999;
config.apiUrl = 'https://evil.example.com';
```

Why it seems fine at first: The variable cannot be reassigned, so it feels protected. Junior developers frequently treat const as a full immutability guarantee.

What breaks in production: Configuration objects, feature flags, and shared constants get mutated across services. In large codebases the mutation happens far from the declaration,

making the bug nearly invisible until a value is wrong at runtime.

The correct pattern:

```
const config: Readonly<{ apiUrl: string; timeout: number }> = {
  apiUrl: 'https://api.example.com',
  timeout: 3000
};

// Now TypeScript catches this at compile time:
// config.timeout = 99999; // Cannot assign to 'timeout' – it is read-only
```

Adding Readonly moves the enforcement to the type system so mutations are caught before the code ships.

Anti-Pattern 2 — Mutating @Input() Objects Inside a Child Component

What it looks like:

```
@Component({
  selector: 'app-user-card',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `{{ user.name }}`
})
export class UserCardComponent {
  @Input() user!: User;

  promote(): void {
    // Directly mutating the input – the parent owns this reference
    this.user.role = 'admin';
    this.user.name = this.user.name.toUpperCase();
  }
}
```

Why it seems fine at first: The mutation appears to work in Default change detection mode. During development with a small dataset it often renders correctly because Angular's default CD cycle catches the change anyway.

What breaks in production: With OnPush the view does not re-render because the reference never changed. More critically, the child is mutating state owned by the parent, creating invisible shared mutable state — a violation of one-way data flow that causes race conditions in larger component trees.

The correct pattern:

```
export class UserCardComponent {
  @Input() user!: User;
  @Output() userChanged = new EventEmitter<User>();

  promote(): void {
    // Produce a new object – never mutate the input
    this.userChanged.emit({
      ...this.user,
      role: 'admin',
      name: this.user.name.toUpperCase()
    });
  }
}
```

The child emits a new object upward; the parent decides whether and how to update state. The input remains read-only from the child's perspective.

Anti-Pattern 3 — Shallow Readonly on Nested Objects (False Safety)

What it looks like:

```
interface AppState {
  readonly user: {
    name: string;
    permissions: string[];
  };
}

const state: AppState = {
  user: { name: 'Alice', permissions: ['read'] }
};

// TypeScript allows both of these – readonly only protected the top-level reference
state.user.name = 'Hacked';
state.user.permissions.push('admin');
```

Why it seems fine at first: The developer added readonly to the interface property and assumes nested data is protected. TypeScript does not raise an error, so the assumption goes unchallenged.

What breaks in production: Nested objects and arrays in state remain fully mutable. In NgRx stores or service-based state, this causes store mutations that bypass reducers or selectors, making state changes untraceable and time-travel debugging unreliable.

The correct pattern:

```
// Use recursive readonly via utility type
type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends object ? DeepReadonly<T[K]> : T[K];
};

interface AppState {
  user: DeepReadonly<{
    name: string;
    permissions: string[];
  }>;
}

// Now TypeScript catches nested mutations at compile time:
// state.user.name = 'Hacked'; // read-only
// state.user.permissions.push(...); // read-only array
```

DeepReadonly recursively applies readonly to every level of the type, making the compile-time protection match the developer's intent.

Section 16 — Mental Model Summary

Topic 14 — readonly & Immutability Patterns | Mental Model

Core Concept in One Sentence:

readonly is a compile-time signal that a property must not be reassigned after initialisation — it does not freeze the object, and it does not protect nested values unless applied recursively.

The 6 Rules:

- 1. const protects the binding; readonly protects the property — they solve different problems
- 2. readonly is erased at runtime — it is a TypeScript-only guarantee
- 3. Readonly and ReadonlyArray are the standard utility types for immutable contracts
- 4. Nested objects require DeepReadonly or Object.freeze() — shallow readonly is silently incomplete
- 5. Never mutate an @Input() — always emit a new object upward through @Output()
- 6. Immutability and OnPush change detection are a pair — neither works reliably without the other

The #1 Angular Rule: Never mutate an object that crosses a component boundary — always produce a new reference and let Angular's change detection do its job.

Quick Reference:

Technique	Scope	Enforced At	Protects Nested?
const	Variable binding	Compile + Runtime	No
readonly (property)	Single property	Compile time only	No
Readonly<T>	All top-level props	Compile time only	No
DeepReadonly<T>	All levels	Compile time only	Yes
Object.freeze()	All top-level props	Runtime	No
Object.freeze() recursive	All levels	Runtime	Yes

The One Thing to Remember:

TypeScript's readonly is a promise, not a lock — it tells your teammates what you intended, but it is the discipline of always spreading or mapping into a new object that actually keeps your Angular state predictable at scale.

TOPIC 15 — Access Modifiers (private, protected, public)

TypeScript | Intermediate | High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 13 — TypeScript Interfaces & Types (access modifiers appear on class members; you need to be comfortable with the distinction between interface contracts and class implementations before modifiers make full sense)
- Topic 14 — readonly & Immutability Patterns (readonly and private are frequently combined in constructor shorthand and service design; understanding one sharpens the other)
- Topic 4 — Arrow Functions & this Binding (access modifiers control what this can reach from outside a class; binding behaviour and visibility are connected in Angular service and component patterns)

Unlocks:

- Topic 16 — TypeScript Generics (generic classes and methods often combine access modifiers with type parameters; being solid on modifiers removes one variable when generics get complex)
- Topic 17 — TypeScript Decorators (Angular decorators such as @Injectable and @Component sit on classes whose members are governed by access modifiers; the two topics compose directly)
- Topic 25 — Component Lifecycle Hooks (lifecycle methods are always public by contract; understanding why, and what should be private alongside them, is a direct application of this topic)
- Topic 26 — @Input() & @Output() in Depth (@Input and @Output properties must be public; knowing that rule and its reason requires a firm grasp of access modifiers first)

Production Readiness Assessment:

After studying access modifiers alone you can write safer, more intentional class designs immediately, but production confidence requires combining this with Topic 17 (Decorators) and Topic 26 (@Input/@Output), because Angular's decorator rules and the component API surface impose specific constraints on which members must be public and which should never be.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "What are the three access modifiers in TypeScript, what does each one do, and what is the default if you don't specify one?"

What a Strong Answer Covers:

- public — accessible from anywhere; the default when no modifier is written
- private — accessible only within the class that declares it; enforced at compile time only
- protected — accessible within the declaring class and any subclass; not accessible from outside the hierarchy
- TypeScript's private is compile-time only; JavaScript's # private field syntax is a true runtime enforcement — they are different things

- In Angular, component template bindings require the member to be public; private members are invisible to the template

What a Weak Answer Misses: Weak answers omit the distinction between TypeScript's compile-time private and JavaScript's native `# private` fields, and rarely mention that Angular templates operate outside the class and therefore cannot access private members.

Question 2 — Mid

The Question: "In an Angular service, when would you choose private over exposing a property or method publicly, and how does that decision affect the components consuming the service?"

What a Strong Answer Covers:

- Internal state (a BehaviorSubject holding raw data) should be private; the public API should expose only a derived observable so consumers cannot push values directly
- Private methods that orchestrate internal logic (mapping, filtering, error normalisation) should never be callable from a component — they are implementation details
- Keeping the public surface small makes the service easier to refactor without breaking consumers
- A common Angular pattern: `private subject$ = new BehaviorSubject([]); public readonly data$ = this.subject$.asObservable()`
- This pattern prevents components from calling `subject$.next()` directly, enforcing one-directional data flow

What a Weak Answer Misses: Weak answers describe what private does without connecting it to the concrete Angular pattern of hiding BehaviorSubjects behind public observable projections, which is the most common real-world application of this decision.

Question 3 — Senior

The Question: "Your team is building a base component class that several feature components will extend. How do you decide which members are private, protected, or public, and what are the risks of getting that boundary wrong?"

What a Strong Answer Covers:

- Members only the base class needs internally — private; this prevents subclasses from depending on implementation details that may change
- Members the base class exposes intentionally to subclasses for customisation — protected; this is the deliberate extension API
- Members the template or external components need — public; Angular's template compiler requires public access
- Getting protected wrong in the base class creates tight coupling — a subclass that relies on a protected implementation detail breaks silently when the base class is refactored
- Getting private wrong (using it when you should use protected) blocks legitimate subclass extension and forces either ugly workarounds or breaking the inheritance model
- The deeper architectural risk: deep inheritance hierarchies in Angular components are almost always the wrong tool — composition via services and content projection is more maintainable; access modifiers on a base component are a design smell if the hierarchy goes beyond one level

What a Weak Answer Misses: Weak answers stay at the definition level and miss the architectural point that inheritance in Angular components is a pattern that should be used sparingly, making the access modifier decisions on a base component class a signal worth examining in the first place.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Everything Public by Default (The Transparent Class)

What it looks like:

```
@Injectable({ providedIn: 'root' })
export class UserService {
  public users$ = new BehaviorSubject<User[]>([]);
  public loading = false;
  public selectedId: number | null = null;

  public fetchUsers(): void { ... }
  public setLoading(val: boolean): void { this.loading = val; }
  public pushUser(user: User): void {
    this.users$.next([...this.users$.getValue(), user]);
  }
}
```

Why it seems fine at first: Everything works. Components can access whatever they need without hitting access errors. It feels pragmatic and avoids any friction during development.

What breaks in production: Any component anywhere in the application can call `users$.next()` directly, call `setLoading()` out of sequence, or mutate `selectedId` without going through the service's intended logic. At scale this produces state that is impossible to trace — the service's internal invariants cannot be enforced, and debugging a corrupted state means auditing every component that touched the service.

The correct pattern:

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private readonly usersSubject$ = new BehaviorSubject<User[]>([]);
  private loading = false;

  // Public API – controlled surface only
  public readonly users$ = this.usersSubject$.asObservable();

  public fetchUsers(): Observable<User[]> {
    this.loading = true;
    return this.http.get<User[]>('/api/users').pipe(
      tap(users => {
        this.usersSubject$.next(users);
        this.loading = false;
      })
    );
  }
}
```

The `BehaviorSubject` is private; components receive a plain `Observable` they can read but cannot push to. All mutations flow through the service's own methods where invariants can be enforced.

Anti-Pattern 2 — Private Members in Component Templates (The Invisible Binding)

What it looks like:

```
@Component({
  selector: 'app-dashboard',
  template: `
<div>{{ userName }}</div>
<button (click)="loadData()">Load</button>
`
})
export class DashboardComponent {
  private userName = 'Alice';
  private loadData(): void { ... }
}
```

Why it seems fine at first: In older versions of Angular and in many IDE setups this appears to work at runtime because TypeScript's private is erased after compilation. The browser sees plain JavaScript with no access restrictions.

What breaks in production: The Angular compiler in strict mode (and Ivy's template type checking) raises errors for private member access in templates. The build fails in CI. Even where it slips through, the intent is violated — the template is external to the class and private was supposed to mean internal only.

The correct pattern:

```
@Component({
  selector: 'app-dashboard',
  template: `
<div>{{ userName }}</div>
<button (click)="loadData()">Load</button>
`
})
export class DashboardComponent {
  // Template-bound members must be public
  public userName = 'Alice';

  // Internal-only logic the template does not call
  private readonly userKey = 'dashboard_user';

  public loadData(): void {
    this.normaliseUser();
  }

  private normaliseUser(): void { ... }
}
```

Members the template binds to are public; members only the class's own methods call are private. The boundary matches both TypeScript's intent and Angular's compiler requirements.

Anti-Pattern 3 — Using protected to Share State Across a Deep Inheritance Chain

What it looks like:

```
export class BaseFormComponent {
  protected formData: Record<string, unknown> = {};
```

```

protected errors: string[] = [];
protected isDirty = false;
}

export class AddressFormComponent extends BaseFormComponent {
protected streetData: string = '';
// Freely reads and writes formData, errors, isDirty from base
}

export class ShippingFormComponent extends AddressFormComponent {
// Three levels deep – reads formData from BaseFormComponent
protected shippingMethod = '';
}

```

Why it seems fine at first: Protected feels like a clean way to share state across related components. The hierarchy looks logical on a class diagram. It avoids prop-drilling and service injection at every level.

What breaks in production: Any change to a protected member in BaseFormComponent ripples invisibly through every subclass. Refactoring is dangerous because there is no clear API contract — subclasses depend on the internal structure of their ancestors. Testing a leaf component requires instantiating the entire chain.

The correct pattern:

```

// Favour composition – shared state lives in a service
@Injectable()
export class FormStateService {
private readonly stateSubject$ = new BehaviorSubject<FormState>(initialState);
public readonly state$ = this.stateSubject$.asObservable();

public patch(partial: Partial<FormState>): void {
this.stateSubject$.next({ ...this.stateSubject$.getValue(), ...partial });
}
}

// Each component is flat, independently testable
@Component({ selector: 'app-shipping-form', ... })
export class ShippingFormComponent {
public state$ = this.formState.state$;
constructor(private formState: FormStateService) {}
}

```

Shared state moves into a service with a controlled public API. Each component is a flat class with no parent dependency, independently instantiable in tests, and unaffected by changes to sibling components.

Section 16 — Mental Model Summary

Topic 15 — Access Modifiers | Mental Model

Core Concept in One Sentence:

Access modifiers are the boundary contract of a class — they declare which parts are internal implementation detail and which parts are intentional API, and in Angular that boundary directly determines what templates, subclasses, and external components are permitted to touch.

The 6 Rules:

- 1. public is the default — always write it explicitly on Angular `@Input()`, `@Output()`, and template-bound members to make intent clear
- 2. private means this class only — Angular's template compiler treats the template as external and will reject private bindings in strict mode
- 3. protected means this class and its subclasses — use it sparingly and only when inheritance is genuinely the right model
- 4. TypeScript's private is compile-time only — JavaScript's `#` syntax is the true runtime equivalent; know the difference
- 5. Hide BehaviorSubjects behind private and expose only public readonly observables — this is the most impactful access modifier pattern in real Angular codebases
- 6. Prefer composition over inheritance — a deep protected chain is almost always a sign that a shared service would be cleaner

The #1 Angular Rule: `@Input()` and `@Output()` properties must always be public — the Angular compiler requires it, and any private or protected decorator binding will fail in strict template type checking.

Quick Reference:

Modifier	Accessible In Class	In Subclass	In Template	Outside
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	No	No
private	Yes	No	No	No
# (JS private)	Yes	No	No	No (runtime)

The One Thing to Remember:

The moment you make a member public in a service or component, you are making a promise to every consumer that it will exist and behave consistently — keep the public surface as small as possible, because every public member you add is a commitment you will eventually have to honour during a refactor.

TOPIC 16 — TypeScript Generics

TypeScript | Intermediate-Advanced | High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 13 — TypeScript Interfaces & Types (generics parameterise interfaces and type aliases; you must be comfortable writing both before adding a type parameter on top)
- Topic 14 — readonly & Immutability Patterns (Readonly, ReadonlyArray, and DeepReadonly are all generic utility types; this topic is where you learn the mechanism behind those patterns)
- Topic 15 — Access Modifiers (generic classes combine type parameters with access modifiers; understanding both separately before combining them prevents confusion)
- Topic 9 — Array Methods (map, filter, reduce are all generic functions under the hood; recognising that connection makes the mental model land faster)

Unlocks:

- Topic 17 — TypeScript Decorators (Angular decorators produce and consume generic types internally; a firm grasp of generics makes decorator signatures readable rather than intimidating)
- Topic 18 — Error Handling (generic Result and Either patterns for typed error handling depend directly on this topic)
- Topic 25 — Component Lifecycle Hooks (Angular's signal APIs and the new typed input() function use generics extensively; this topic is the prerequisite)
- Topic 26 — @Input() & @Output() in Depth (OutputEmitterRef and the new signal-based input() are generic; you cannot read their type signatures without this topic)

Production Readiness Assessment:

After studying generics alone you can write reusable typed utilities and understand Angular's own type signatures immediately, but production confidence requires combining this with Topic 17 (Decorators) and Topic 26 (@Input/@Output), because the places where generics matter most in Angular — typed event emitters, generic services, and the new signal input API — appear at the component boundary where those topics live.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "What is a generic in TypeScript and why would you use one instead of using the any type?"

What a Strong Answer Covers:

- A generic is a type parameter that lets a function, class, or interface operate on a type that is specified by the caller rather than hard-coded by the author
- any disables type checking entirely — the compiler stops tracking what the value is
- A generic preserves the relationship between input and output types — if you pass in a string[], you get back a string[], not an unknown[]
- Generics enable reuse without sacrificing safety; any enables reuse by abandoning safety

- Concrete Angular example: `HttpClient.get()` — the generic tells the compiler what shape the response will be so the consuming component gets full type checking on the result

What a Weak Answer Misses: Weak answers define generics correctly but fail to contrast them with any in terms of what type information is preserved versus discarded, and do not give a concrete example of where the loss of any shows up as a real bug.

Question 2 — Mid

The Question: "How would you design a generic Angular service that can handle API responses for any resource type, and how would you use constraints to make it safer?"

What a Strong Answer Covers:

- A generic base service accepts a type parameter `T` representing the resource shape: `class ApiService`
- Constraints (extends) restrict what `T` can be — for example `T extends { id: number }` guarantees the service can safely reference `item.id` without a runtime error
- The service's methods return `Observable`, `Observable`, and take `T` or `Partial` as parameters — all typed through to the caller
- Concrete methods: `getAll(): Observable`, `getById(id: number): Observable`, `create(payload: Omit): Observable`
- The constraint prevents the service being instantiated with a primitive type or a shape that lacks the fields the service's logic depends on

What a Weak Answer Misses: Weak answers write a generic service with a type parameter but omit constraints entirely, producing a service that is technically generic but will produce cryptic runtime errors when `T` lacks the properties the implementation accesses.

Question 3 — Senior

The Question: "Your team wants to build a generic form state manager that works with any reactive form value shape, enforces that fields can only be patched with valid keys from that shape, and exposes a typed signal for the current value. What TypeScript features would you combine and what are the trade-offs?"

What a Strong Answer Covers:

- `keyof T` constrains patch methods so only valid property names of `T` are accepted — eliminates a whole class of typo-driven runtime bugs
- `Partial` allows patching a subset of fields without requiring the full object
- A generic signal: `ValueSignal = signal(initialValue)` gives fully typed reads and writes
- Mapped types (`Record`) let you build a parallel validation state object that structurally mirrors the form shape without manual duplication
- Trade-off 1: complexity — a heavily generic form manager is harder for junior developers to extend; overly clever types slow down the TypeScript compiler on large forms
- Trade-off 2: Angular's own `FormGroup` is already generic in Angular 14+; building a parallel abstraction risks diverging from the framework's own typed forms API rather than extending it
- The senior architectural judgement: use typed `FormGroup` generics where Angular provides them natively; reach for a custom generic state manager only for non-form state that genuinely needs the same pattern

What a Weak Answer Misses: Weak answers describe keyof and Partial correctly but miss the trade-off between custom generic abstractions and Angular's own typed reactive forms API, which is the architectural decision that actually matters in this scenario.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Using any as a Generic Escape Hatch

What it looks like:

```
// Generic-looking but type-safety is completely gone
@Injectable({ providedIn: 'root' })
export class DataService {
  getData(endpoint: string): Observable<any> {
    return this.http.get<any>(endpoint);
  }
}

// In a component – no errors, but no safety either
this.dataService.getData('/api/users').subscribe(data => {
  console.log(data.naem); // typo – no compiler error, silent runtime bug
});
```

Why it seems fine at first: The service is flexible — it handles any endpoint without needing a separate service per resource. TypeScript never complains. Development is fast.

What breaks in production: Every consumer is on their own for type safety. Typos in property access, wrong assumptions about response shape, and missing null checks all become silent runtime failures. The service offers the illusion of a typed API while providing none of the guarantees.

The correct pattern:

```
@Injectable({ providedIn: 'root' })
export class DataService {
  getData<T>(endpoint: string): Observable<T> {
    return this.http.get<T>(endpoint);
  }
}

// In a component – the type flows through
this.dataService.getData<User[]>('/api/users').subscribe(users => {
  console.log(users[0].naem); // TypeScript error – 'naem' does not exist on User
});
```

The type parameter T is supplied by the caller, flows through HttpClient's own generic, and the compiler now checks every property access on the result.

Anti-Pattern 2 — Unconstrained Generics Accessing Properties That May Not Exist

What it looks like:

```
// Looks generic and reusable
function getLabel<T>(item: T): string {
  return item.label; // TypeScript error – T might not have 'label'
}

// Developer "fixes" it by casting
function getLabel<T>(item: T): string {
```

```
return (item as any).label; // compiles – but safety is gone
}
```

Why it seems fine at first: The function is called with objects that always happen to have a label property during development. The cast silences the error and everything appears to work.

What breaks in production: When the function is reused with a different type — a stripped-down DTO, an API response with a different shape, a refactored interface that renamed label to displayName — the runtime error appears with no compile-time warning. The cast hid the problem for months.

The correct pattern:

```
// Constrain T to guarantee the property exists
interface Labelled {
  label: string;
}

function getLabel<T extends Labelled>(item: T): string {
  return item.label; // Safe – T is guaranteed to have label
}

// In Angular – a reusable dropdown that works with any option type
@Component({ selector: 'app-dropdown', template: `...` })
export class DropdownComponent<T extends Labelled> {
  @Input() options: T[] = [];
  @Input() selected: T | null = null;
}
```

The extends constraint narrows what T can be, so the compiler can verify every property access without any cast.

Anti-Pattern 3 — Duplicating Typed Services Instead of Writing a Generic Base

What it looks like:

```
// Three services – identical logic, different types
@Injectable({ providedIn: 'root' })
export class UserService {
  getAll(): Observable<User[]> { return this.http.get<User[]>('/api/users'); }
  getById(id: number): Observable<User> { return
    this.http.get<User>(`/api/users/${id}`); }
}

@Injectable({ providedIn: 'root' })
export class ProductService {
  getAll(): Observable<Product[]> { return this.http.get<Product[]>('/api/products'); }
  getById(id: number): Observable<Product> { return
    this.http.get<Product>(`/api/products/${id}`); }
}

// A third for Orders... and a fourth for Invoices...
```

Why it seems fine at first: Each service is explicit and easy to read. There is no abstraction to understand. Copy-paste is fast.

What breaks in production: A bug in the error handling logic, a change to the base URL pattern, or an addition of a loading state has to be applied to every duplicated service. In a

codebase with twelve resource types, one missed update creates an inconsistency that is found only in production.

The correct pattern:

```
// One generic base service
export class ResourceService<T extends { id: number }> {
  constructor(protected http: HttpClient, protected endpoint: string) {}

  getAll(): Observable<T[]> { return this.http.get<T[]>(this.endpoint); }
  getById(id: number): Observable<T> { return
    this.http.get<T>(`${this.endpoint}/${id}`); }
  create(payload: Omit<T, 'id'>): Observable<T> {
    return this.http.post<T>(this.endpoint, payload);
  }
}

// Each concrete service is a one-liner
@Injectable({ providedIn: 'root' })
export class UserService extends ResourceService<User> {
  constructor(http: HttpClient) { super(http, '/api/users'); }
}

@Injectable({ providedIn: 'root' })
export class ProductService extends ResourceService<Product> {
  constructor(http: HttpClient) { super(http, '/api/products'); }
}
```

Bug fixes and enhancements to the shared logic happen once in `ResourceService` and propagate to every concrete service automatically.

Section 16 — Mental Model Summary

Topic 16 — TypeScript Generics | Mental Model

Core Concept in One Sentence:

A generic is a placeholder type that is filled in by the caller — it lets you write logic once and have the TypeScript compiler verify it correctly for every specific type it is used with, without sacrificing any type information along the way.

The 6 Rules:

- 1. Use a generic when the logic is the same but the type varies — if you find yourself writing the same function twice for different types, a generic is the answer
- 2. Always constrain generics with `extends` when your implementation accesses specific properties — an unconstrained generic that accesses a property is either a compiler error or a hidden cast
- 3. Let callers infer the type where possible — `HttpClient.get()` is cleaner than spelling it out everywhere when inference is available
- 4. Angular's own APIs are deeply generic — `HttpClient`, `EventEmitter`, `signal()`, and the new `input()` are all generics; reading their signatures is a core Angular skill
- 5. Prefer generic base services over copy-pasted typed services — one fix in the base propagates everywhere

- 6. Do not over-engineer — if a function is only ever called with one type, a generic adds complexity without benefit; use a concrete type instead

The #1 Angular Rule: When using HttpClient, always supply the generic type parameter — `http.get()` — because without it the response is typed as `Object`, and every property access downstream loses compiler protection.

Quick Reference:

Pattern	Syntax	Angular Use Case
Generic function	<code>fn<T>(arg: T): T</code>	Utility helpers, pipe operators
Generic class	<code>class Service<T></code>	Base resource services
Generic interface	<code>interface Repo<T></code>	Repository contracts
Constraint	<code><T extends Base></code>	Services needing id, label, etc.
Utility type	<code>Partial<T></code> , <code>Readonly<T></code>	Form patching, state contracts
Inferred generic	<code>http.get<User[]>()</code>	HttpClient calls
keyof constraint	<code><K extends keyof T></code>	Typed patch/select methods

The One Thing to Remember:

A generic is not magic — it is a variable for a type, and like any variable it is only as useful as the constraints you put around it; an unconstrained generic that accesses properties is just any with extra steps, so always ask yourself what you need T to guarantee before you write the function body.

TOPIC 17 — TypeScript Decorators

TypeScript / Angular | Intermediate-Advanced | High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 13 — TypeScript Interfaces & Types (decorator factories return functions that operate on typed targets; reading decorator signatures requires comfort with interface and type syntax)
- Topic 15 — Access Modifiers (decorators in Angular are applied to class members whose visibility matters — `@Input()` must be public, `@ViewChild()` results are typically private; the two topics are inseparable in practice)
- Topic 16 — TypeScript Generics (Angular's decorator APIs produce and consume generic types — `EventEmitter`, `QueryList`, `signal()`; decorator signatures are unreadable without generics)
- Topic 4 — Arrow Functions & this Binding (decorators that wrap methods must handle this correctly; method decorators that use regular functions versus arrow functions behave differently inside the wrapper)

Unlocks:

- Topic 25 — Component Lifecycle Hooks (every lifecycle hook exists on a class decorated with `@Component`; understanding what `@Component` does to a class gives lifecycle hooks their full context)
- Topic 26 — `@Input()` & `@Output()` in Depth (`@Input()` and `@Output()` are property decorators; this topic is the direct prerequisite for understanding how Angular reads those decorators at bootstrap time to wire the component API)

Production Readiness Assessment:

After studying TypeScript decorators alone you will understand what Angular's built-in decorators do and how to read their signatures confidently, but you will not be ready to write custom decorators for production use without also studying Topic 25 and Topic 26, because the most valuable custom decorator patterns in Angular all depend on the component lifecycle and the input/output binding system.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "What is a decorator in TypeScript and what are the four types Angular uses?"

What a Strong Answer Covers:

- A decorator is a special function prefixed with `@` that is applied to a class, method, property, or parameter and can read or modify its target at the point of class definition
- TypeScript decorators are a stage-3 ECMAScript proposal; Angular uses its own decorator implementation compiled away by the Angular compiler (ngc) — they are not standard JavaScript at runtime
- Class decorators: `@Component`, `@Directive`, `@Injectable`, `@NgModule`, `@Pipe` — attach metadata to a class

- Property decorators: `@Input()`, `@Output()`, `@ViewChild()`, `@ViewChildren()`, `@HostBinding()` — attach metadata to a class member
- Method decorators: `@HostListener()` — wrap or annotate a method
- Parameter decorators: `@Inject()`, `@Optional()`, `@Self()`, `@SkipSelf()` — annotate constructor parameters for the dependency injection system
- Angular's compiler reads all of this metadata at build time and uses it to generate the component factory, the change detection instructions, and the DI graph

What a Weak Answer Misses: Weak answers list the four decorator types without explaining that Angular's compiler processes the metadata at build time — leaving the interviewer with the impression the candidate thinks decorators are a runtime feature that Angular inspects dynamically.

Question 2 — Mid

The Question: "How does Angular's dependency injection system use the `@Injectable()` decorator, and what is the difference between `providedIn: 'root'` and providing a service in a specific module or component?"

What a Strong Answer Covers:

- `@Injectable()` tells Angular's compiler that this class participates in the DI system and may have its own constructor dependencies that need to be resolved
- `providedIn: 'root'` registers the service in the root injector — one singleton instance for the entire application, tree-shakeable (if nothing injects it, it is not bundled)
- Providing in an `NgModule`'s `providers` array registers the service in that module's injector — shared across all components in that module but not tree-shakeable in the same way
- Providing in a component's `providers` array creates a new instance scoped to that component and its children — destroyed when the component is destroyed; useful for stateful services that should not be singletons
- The practical Angular decision: use `providedIn: 'root'` for stateless or application-wide services; use component-level providers for services that hold component-scoped state (a form wizard, a multi-step flow)

What a Weak Answer Misses: Weak answers describe `providedIn: 'root'` correctly but treat it as the only option, missing the component-level provider pattern which is the most architecturally significant distinction.

Question 3 — Senior

The Question: "A senior developer on your team proposes writing a custom `@AutoUnsubscribe()` class decorator that automatically calls `unsubscribe` on all `Subscription` properties when a component is destroyed. What are the benefits, the risks, and would you approve it for production use?"

What a Strong Answer Covers:

- The appeal: eliminates boilerplate `ngOnDestroy` methods and subscription cleanup across every component; a real pain point in large Angular codebases
- Implementation risk 1: the decorator must hook into `ngOnDestroy` — if the component already defines `ngOnDestroy`, the decorator must compose with it, not replace it; getting this wrong silently breaks cleanup

- Implementation risk 2: the decorator iterates over class properties at runtime using reflection or property enumeration, which is fragile — it can miss subscriptions stored in arrays, Maps, or created after construction
- Implementation risk 3: it obscures what is happening — a developer reading the component class sees no `ngOnDestroy` and no `takeUntilDestroyed`, and incorrectly concludes there is a memory leak
- The Angular-idiomatic alternative: the `takeUntilDestroyed()` operator (Angular 16+) with `inject(DestroyRef)` achieves the same result with zero reflection, full compiler support, and visible intent at the point of the subscription
- Production verdict: reject the custom decorator; adopt `takeUntilDestroyed()` — it solves the same problem with framework support, no reflection, and readable code

What a Weak Answer Misses: Weak answers evaluate the decorator only on its happy path and miss the composition risk with existing `ngOnDestroy` implementations, and crucially do not know that Angular 16 introduced a first-party solution that makes the custom decorator unnecessary.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Skipping `@Injectable()` on a Service That Has Dependencies

What it looks like:

```
// No @Injectable() decorator
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
}

// Provided in module – seems fine
@NgModule({
  providers: [UserService]
})
export class UserModule {}
```

Why it seems fine at first: Angular can sometimes resolve the service without the decorator in non-strict, non-Ivy builds. The application works in development. The omission is invisible until a specific build configuration exposes it.

What breaks in production: With Ivy and ahead-of-time (AOT) compilation — which is the default for all production builds — the compiler cannot emit the correct dependency injection metadata without `@Injectable()`. The production build either throws a runtime error or silently injects undefined for `HttpClient`. This bug appears only in production AOT builds, not in development JIT mode.

The correct pattern:

```
// Always decorate – no exceptions
@Injectable({
  providedIn: 'root'
})
export class UserService {
  constructor(private http: HttpClient) {}
```

```

getUsers(): Observable<User[]> {
  return this.http.get<User[]>('/api/users');
}
}

```

`@Injectable()` is mandatory on any class that either has constructor dependencies or may be extended by a class that does. The rule is: if it is a service, it gets the decorator — always.

Anti-Pattern 2 — Using `@HostListener()` for Events That Belong in the Template

What it looks like:

```

@Component({
  selector: 'app-button',
  template: `<button>Click me</button>`
})
export class ButtonComponent {
  @HostListener('click', ['$event'])
  onClick(event: MouseEvent): void {
    console.log('clicked', event);
  }

  @HostListener('window:resize', ['$event'])
  onResize(event: Event): void {
    this.updateLayout();
  }
}

```

Why it seems fine at first: `@HostListener()` is an official Angular API. Using it for both the host element click and the window resize event feels consistent. Everything is in the class definition, away from the template.

What breaks in production: `@HostListener()` on window or document events is never automatically unsubscribed — the listener stays active after the component is destroyed, causing memory leaks and ghost event handlers in long-running single-page applications.

The correct pattern:

```

@Component({
  selector: 'app-button',
  host: { '(click)': 'onClick($event)' },
  template: `<button>Click me</button>`
})
export class ButtonComponent implements OnInit {
  onClick(event: MouseEvent): void {
    console.log('clicked', event);
  }

  // Global events: use fromEvent with takeUntilDestroyed
  constructor() {
    fromEvent(window, 'resize')
      .pipe(takeUntilDestroyed())
      .subscribe(() => this.updateLayout());
  }
}

```

```
private updateLayout(): void { ... }
}
```

Host element events go in the host metadata or template; global events use `fromEvent` with `takeUntilDestroyed()` so cleanup is guaranteed and the subscription lifetime is explicit.

Anti-Pattern 3 — Applying `@ViewChild()` Without Handling the Timing of Availability

What it looks like:

```
@Component({
  selector: 'app-chart',
  template: `<canvas #chartCanvas></canvas>`
})
export class ChartComponent implements OnInit {
  @ViewChild('chartCanvas') canvas!: ElementRef<HTMLCanvasElement>;

  ngOnInit(): void {
    // canvas is undefined here – the view has not been initialised yet
    const ctx = this.canvas.nativeElement.getContext('2d');
    this.renderChart(ctx);
  }
}
```

Why it seems fine at first: `ngOnInit` feels like the natural place to set up a component. The `@ViewChild()` property is declared and typed. TypeScript shows no errors. In some timing scenarios with simple templates it may even work intermittently.

What breaks in production: `@ViewChild()` references are not populated until after `ngAfterViewInit`. Accessing `this.canvas` in `ngOnInit` reliably returns undefined — the non-null assertion (!) suppresses the TypeScript warning but guarantees a runtime `TypeError` in production.

The correct pattern:

```
@Component({
  selector: 'app-chart',
  template: `<canvas #chartCanvas></canvas>`
})
export class ChartComponent implements AfterViewInit {
  @ViewChild('chartCanvas') canvas!: ElementRef<HTMLCanvasElement>;

  ngAfterViewInit(): void {
    // canvas is guaranteed to be available here
    const ctx = this.canvas.nativeElement.getContext('2d');
    this.renderChart(ctx);
  }

  // For conditional templates – use the setter pattern
  @ViewChild('chartCanvas')
  set chartCanvas(el: ElementRef<HTMLCanvasElement> | undefined) {
    if (el) {
      const ctx = el.nativeElement.getContext('2d');
      this.renderChart(ctx);
    }
  }
}
```

```
}
```

ViewChild references are read in `ngAfterViewInit`, not `ngOnInit`. For conditionally rendered elements, the setter pattern fires precisely when the element enters or leaves the DOM.

Section 16 — Mental Model Summary

Topic 17 — TypeScript Decorators | Mental Model

Core Concept in One Sentence:

A decorator is a metadata annotation that Angular's compiler reads at build time to understand how a class, property, method, or parameter should be wired into the framework — it is not magic at runtime, it is instructions for the compiler.

The 6 Rules:

- 1. Every Angular class that participates in the framework must have a class decorator — `@Component`, `@Directive`, `@Injectable`, `@Pipe`, or `@NgModule` — with no exceptions in AOT/Ivy builds
- 2. `@Injectable()` is required on any service with constructor dependencies, even if it seems to work without it in development JIT mode
- 3. `@Input()` and `@Output()` properties must always be public — the decorator does not override the access modifier requirement
- 4. `@ViewChild()` and `@ContentChild()` results are not available until `ngAfterViewInit` and `ngAfterContentInit` respectively — never access them in `ngOnInit`
- 5. Global event listeners attached via `@HostListener()` are not automatically unsubscribed — always pair them with a cleanup strategy or replace with `fromEvent` + `takeUntilDestroyed()`
- 6. Prefer Angular's built-in decorators and first-party APIs (`takeUntilDestroyed`, the new signal inputs) over custom decorators that use runtime reflection

The #1 Angular Rule: `@ViewChild()` results are undefined in `ngOnInit` — always access them in `ngAfterViewInit`, and use the setter pattern for elements inside conditional blocks.

Quick Reference:

Decorator	Type	Available From	Common Mistake
<code>@Component()</code>	Class	Build time	Missing required metadata fields
<code>@Injectable()</code>	Class	Build time	Omitting when dependencies exist
<code>@Input()</code>	Property	Parent binding	Making it non-public
<code>@Output()</code>	Property	Parent binding	Wrong EventEmitter generic type
<code>@ViewChild()</code>	Property	<code>ngAfterViewInit</code>	Accessing in <code>ngOnInit</code>
<code>@ContentChild()</code>	Property	<code>ngAfterContentInit</code>	Accessing in <code>ngOnInit</code>
<code>@HostListener()</code>	Method	Event fires	Missing unsubscribe for global events
<code>@HostBinding()</code>	Property	Change detection	Overusing instead of class bindings

The One Thing to Remember:

Angular decorators are compile-time instructions, not runtime behaviour — when something breaks mysteriously only in a production AOT build and works fine in development, the first place to look is a missing or misconfigured decorator, because the JIT compiler is forgiving where the

AOT compiler is strict.

TOPIC 18 — Error Handling

Angular / RxJS | Intermediate | High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 8 — Promises & async/await (try/catch for async errors is the direct prerequisite; error handling in Angular services extends the same mental model into the Observable world)
- Topic 16 — TypeScript Generics (typed error results — Result, Either patterns — are generic types; this topic is the mechanism behind safe error modelling)
- Topic 17 — TypeScript Decorators (@Injectable services are where most Angular error handling logic lives; understanding how services are constructed and scoped is necessary context)
- Topic 6 — Callback Functions (RxJS error handling operators — catchError, retry, retryWhen — accept callback functions; the pattern is familiar but the Observable context adds constraints)

Unlocks:

- Topic 19 — Optional Chaining & Nullish Coalescing (optional chaining is a defensive error-avoidance pattern; studying error handling first gives it proper motivation)
- Topic 25 — Component Lifecycle Hooks (error states need to be surfaced in the view; knowing when and how to update component state from an error path requires understanding the lifecycle)
- Topic 26 — @Input() & @Output() in Depth (error events emitted through @Output() are a standard pattern for bubbling errors from child to parent)

Production Readiness Assessment:

After studying error handling alone you can write defensively structured Angular services and handle HTTP errors correctly in isolation, but production confidence requires combining this with Topic 19 (Optional Chaining) and Topic 25 (Lifecycle Hooks), because production error handling in Angular spans the full chain from HTTP layer through service through component state to user-visible feedback — and that chain touches all three topics.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "How do you handle HTTP errors in an Angular service using RxJS, and what is the role of catchError?"

What a Strong Answer Covers:

- HTTP errors in Angular arrive as error notifications on the Observable stream — they do not throw in the traditional sense; they terminate the stream with an error
- catchError intercepts the error notification and must return a new Observable — either a fallback value via of(), a re-thrown error via throwError(), or a transformed error
- The error object from HttpClient is an HttpResponse — it carries status, statusText, error (the response body), and url
- Placing catchError in the service rather than the component keeps error-handling logic centralised and prevents every component from having to know about HTTP status codes

- A strong pattern: map known status codes to domain errors inside `catchError`, and let unknown errors propagate so a global error handler can catch them

What a Weak Answer Misses: Weak answers describe `catchError` as "catching the error" without explaining that it must return an Observable, which means returning `throwError()` to re-propagate is functionally different from returning `of(null)` as a fallback — and choosing the wrong one silently swallows errors that should reach a global handler.

Question 2 — Mid

The Question: "What is the difference between handling errors inside a `catchError` in a service versus using an Angular `ErrorHandler`, and when would you use each?"

What a Strong Answer Covers:

- `catchError` in a service is local and recoverable — it intercepts a specific Observable stream, handles known errors (404 -> empty array, 401 -> redirect to login), and returns a new Observable that the component can use normally
- Angular's `ErrorHandler` is a global last-resort handler — it catches uncaught errors from anywhere in the application including errors that escape all local handlers, component errors, and errors thrown outside Observable streams
- The default `ErrorHandler` just logs to the console; replacing it with a custom implementation lets you send errors to a monitoring service (Sentry, Datadog) in production
- The correct layered architecture: `catchError` in services for recoverable domain errors, a global `ErrorHandler` for unrecoverable errors and observability
- HTTP interceptors sit between the two — they are the right place for cross-cutting concerns like adding retry logic for 503 errors or redirecting on 401 without modifying every service

What a Weak Answer Misses: Weak answers describe one of the two mechanisms without articulating that they are complementary layers with different responsibilities — treating them as alternatives rather than as a stack.

Question 3 — Senior

The Question: "You are designing the error handling architecture for a large Angular application with twenty feature modules, each making independent HTTP calls. How would you structure error handling so that local recoverable errors are handled close to their source, unrecoverable errors reach a monitoring service, and the user always sees a meaningful message — without duplicating logic across every service?"

What a Strong Answer Covers:

- Layer 1 — HTTP Interceptor: a single interceptor handles cross-cutting HTTP concerns — retry on 503 with exponential backoff, redirect on 401, attach correlation IDs for server-side tracing; no service needs to implement this itself
- Layer 2 — Service-level `catchError`: each service catches domain-specific recoverable errors and maps them to typed domain error objects or safe fallback values
- Layer 3 — Global `ErrorHandler`: replaces Angular's default handler; sends unhandled errors to a monitoring service with context (user ID, route, Angular version)
- Layer 4 — Component error state: components bind to an `error$` signal or observable; a shared `ErrorMessageComponent` renders user-friendly messages from typed domain errors

- The typed domain error pattern: a discriminated union (type `AppError = NetworkError | AuthError | ValidationError | NotFoundError`) lets the component switch on `error.type` and render the correct message without string matching
- Trade-off: this architecture requires upfront investment; for a small application a simpler two-layer approach is sufficient and the full four-layer version is over-engineering

What a Weak Answer Misses: Weak answers propose either a single global handler (too coarse) or per-service `catchError` everywhere (cross-cutting concerns duplicated in twenty services), missing that the correct answer is a deliberate layered architecture where each layer has a single, well-defined responsibility.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Swallowing Errors Silently with an Empty `catchError`

What it looks like:

```
@Injectable({ providedIn: 'root' })
export class OrderService {
  getOrders(): Observable<Order[]> {
    return this.http.get<Order[]>('/api/orders').pipe(
      catchError(() => {
        // Error is caught – and then nothing happens
        return of([]);
      })
    );
  }
}

// In the component – the list is just empty, no indication of failure
this.orders$ = this.orderService.getOrders();
```

Why it seems fine at first: The application does not crash. The component renders an empty list rather than an error state. Users do not see a red error screen. It feels defensive.

What breaks in production: Every HTTP failure — server down, authentication expired, network timeout, malformed response — is silently converted into an empty array. The user sees a blank list with no explanation. No error reaches the monitoring service. The on-call engineer has no signal that anything is wrong. A 401 that should redirect to login is swallowed and the user sits on a broken authenticated page.

The correct pattern:

```
@Injectable({ providedIn: 'root' })
export class OrderService {
  getOrders(): Observable<Order[] | AppError> {
    return this.http.get<Order[]>('/api/orders').pipe(
      catchError((err: HttpResponse) => {
        // Map known errors to typed domain errors
        if (err.status === 404) {
          return of({ type: 'NOT_FOUND' } as AppError);
        }
        if (err.status === 401) {
          this.router.navigate(['/login']);
          return EMPTY;
        }
        // Unknown errors propagate to the global ErrorHandler
      })
    );
  }
}
```

```

return throwError(() => err);
})
);
}
}

```

Known errors are mapped to typed domain values the component can act on. Unknown errors propagate to the global handler where they are logged and monitored. Nothing is silently discarded.

Anti-Pattern 2 — Handling HTTP Errors in Every Component Instead of Centralising

What it looks like:

```

// Repeated in UserComponent, OrderComponent, ProductComponent...
export class UserListComponent implements OnInit {
  ngOnInit(): void {
    this.http.get<User[]>('/api/users').subscribe({
      next: (users) => { this.users = users; },
      error: (err: HttpResponse) => {
        if (err.status === 401) {
          this.router.navigate(['/login']); // duplicated everywhere
        } else if (err.status === 503) {
          setTimeout(() => this.ngOnInit(), 3000); // duplicated everywhere
        } else {
          this.error = 'Something went wrong';
        }
      }
    });
  }
}

```

Why it seems fine at first: It is explicit. Every component handles its own errors. No abstraction is needed. A developer can read the component and understand exactly what happens on failure.

What breaks in production: The 401 redirect logic, the 503 retry logic, and the error message string are duplicated across every component that makes an HTTP call. When the auth redirect destination changes, every component must be updated. One inconsistency in the 401 handling leaves users stuck on authenticated pages after session expiry.

The correct pattern:

```

// One interceptor – handles cross-cutting HTTP error concerns once
@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
  constructor(private router: Router) {}

  intercept(req: HttpRequest<unknown>, next: HttpHandler):
    Observable<HttpEvent<unknown>> {
    return next.handle(req).pipe(
      retry({ count: 2, delay: 1000, resetOnSuccess: true }),
      catchError((err: HttpResponse) => {
        if (err.status === 401) {
          this.router.navigate(['/login']);
        }
        return EMPTY;
      })
    );
  }
}

```

```

}
return throwError(() => err);
})
);
}
}

```

Cross-cutting HTTP error behaviour lives once in the interceptor. Components remain clean and only handle the domain-specific cases that genuinely vary between them.

Anti-Pattern 3 — Using try/catch with subscribe() Instead of RxJS Error Operators

What it looks like:

```

export class DashboardComponent implements OnInit {
  ngOnInit(): void {
    try {
      this.dataService.getData().subscribe(data => {
        this.processData(data);
      });
    } catch (error) {
      // This never catches Observable errors
      console.error('Error fetching data:', error);
      this.showError = true;
    }
  }
}

```

Why it seems fine at first: try/catch is the familiar JavaScript error handling pattern. The developer knows it works with async/await and applies the same mental model to subscribe(). TypeScript raises no error. It looks correct.

What breaks in production: try/catch cannot catch errors emitted asynchronously through an Observable stream. The subscribe() call returns synchronously — the try block exits before the HTTP request completes. When the Observable errors, the error notification arrives later on the event loop with no surrounding try/catch, reaching Angular's default ErrorHandler while this.showError remains false and the component shows no feedback to the user.

The correct pattern:

```

export class DashboardComponent implements OnInit {
  showError = false;

  ngOnInit(): void {
    this.dataService.getData().pipe(
      catchError(err => {
        this.showError = true;
        return EMPTY;
      })
    ).subscribe(data => {
      this.processData(data);
    });

    // Or use the error callback in subscribe directly
    this.dataService.getData().subscribe({
      next: (data) => this.processData(data),

```

```

error: (err) => {
  this.showError = true;
  console.error(err);
}
});
}
}

```

Observable errors must be handled on the Observable error path — either via `catchError` in the pipe or via the error callback in `subscribe`. `try/catch` is the correct pattern only when using `async/await` with `lastValueFrom()` or `firstValueFrom()`.

Section 16 — Mental Model Summary

Topic 18 — Error Handling | Mental Model

Core Concept in One Sentence:

Error handling in Angular is a layered system — HTTP interceptors catch cross-cutting concerns, services catch and type domain errors, components display meaningful state, and a global `ErrorHandler` catches everything that escapes, so no error is ever silently lost and no cross-cutting logic is ever duplicated.

The 6 Rules:

- 1. `catchError` must return an Observable — `of()` for a fallback value, `EMPTY` to complete silently, `throwError()` to re-propagate, never just `void`
- 2. `try/catch` does not catch Observable errors — use `catchError` in the pipe or the error callback in `subscribe`
- 3. Silent `catchError` that returns `of(fallback)` with no logging is almost always wrong — something must know the error occurred
- 4. Cross-cutting HTTP concerns (401 redirect, retry, correlation IDs) belong in an HTTP interceptor — not in individual services or components
- 5. Map known HTTP errors to typed domain errors in the service layer — components should switch on `error.type`, not on HTTP status codes
- 6. Replace Angular's default `ErrorHandler` in every production application — the default only logs to the console; production needs monitoring

The #1 Angular Rule: Never subscribe to an Observable in a component without an error callback or a `catchError` in the pipe — an unhandled Observable error will reach Angular's default `ErrorHandler`, log to the console, and leave the component in a broken state with no user feedback.

Quick Reference:

Scenario	Correct Tool	Wrong Tool
Recover from HTTP error in service	<code>catchError -> of()</code> or typed error	<code>try/catch</code> around <code>subscribe</code>
Retry failed HTTP requests	<code>retry()</code> operator in interceptor	<code>setTimeout</code> in component
Redirect on 401 globally	HTTP Interceptor <code>catchError</code>	Per-service <code>catchError</code>
Send errors to monitoring	Custom <code>ErrorHandler</code>	<code>console.error</code> everywhere
Handle <code>async/await</code> errors	<code>try/catch</code>	<code>catchError</code> (wrong context)

Type-safe error modelling	Discriminated union <code>AppError</code>	String error messages
Unhandled errors last resort	Global <code>ErrorHandler</code>	Uncaught promise/Observable

The One Thing to Remember:

An error that is caught but not reported is worse than an error that crashes visibly — a crash tells you something is wrong, a silent swallow lets a broken application limp along while users silently give up and leave.

TOPIC 19 — Optional Chaining & Nullish Coalescing

TypeScript | Beginner-Intermediate | High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 1 — `let / const / var` (understanding variable declarations and their default undefined state is the baseline motivation for why optional chaining exists)
- Topic 13 — TypeScript Interfaces & Types (optional properties marked with `?` in interfaces are the primary source of the nullable types that optional chaining and nullish coalescing defend against)
- Topic 18 — Error Handling (optional chaining is a defensive error-avoidance pattern — studying error handling first gives it full motivation; you reach for `?.` precisely because you have already seen what uncaught `TypeError`s cost in production)
- Topic 16 — TypeScript Generics (generic utility types like `Partial` make every property optional; understanding why `Partial` exists explains why nullable access patterns are so common in Angular service and form code)

Unlocks:

- Topic 25 — Component Lifecycle Hooks (component state is frequently undefined before data arrives; optional chaining in templates and lifecycle methods is a direct application of this topic in that context)
- Topic 26 — `@Input()` & `@Output()` in Depth (`@Input()` properties are undefined before the parent binds them; safe access patterns using `?.` and `??` are essential for writing robust component APIs)

Production Readiness Assessment:

After studying optional chaining and nullish coalescing alone you can write defensively safe property access and default value patterns immediately — this is one of the few topics that is genuinely production-ready in isolation, though combining it with Topic 25 (Lifecycle Hooks) and Topic 26 (`@Input/@Output`) unlocks the full set of Angular-specific contexts where these operators appear most frequently.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "What is the difference between optional chaining (`?.`) and the logical OR operator (`||`) for providing default values, and when does the distinction matter?"

What a Strong Answer Covers:

- `||` returns the right-hand side when the left-hand side is any falsy value — including `0`, empty string `"`, and `false` — which are legitimate values in many Angular contexts
- `??` (nullish coalescing) returns the right-hand side only when the left-hand side is `null` or `undefined` — it treats `0`, `"`, and `false` as valid values and does not replace them
- Concrete Angular example: a component receives a `@Input()` `count` that could legitimately be `0`; using `count || 10` silently replaces `0` with `10`, breaking the display; `count ?? 10` correctly renders `0`

- `?.` short-circuits property access — if the left side is null or undefined the entire expression evaluates to undefined rather than throwing a `TypeError`
- The two operators compose: `user?.profile?.displayName ?? 'Anonymous'` safely traverses a nullable chain and provides a typed default only for null or undefined

What a Weak Answer Misses: Weak answers treat `||` and `??` as interchangeable and do not give a concrete example where a legitimate falsy value (`0`, `false`, `''`) would be incorrectly replaced by `||`, which is exactly the class of silent bug these operators exist to prevent.

Question 2 — Mid

The Question: "How does Angular's template syntax interact with optional chaining, and what are the alternatives Angular provides for handling nullable values in templates?"

What a Strong Answer Covers:

- Angular templates support `?.` directly — `{{ user?.profile?.displayName }}` is valid template syntax and short-circuits to undefined (rendered as empty) rather than throwing
- The `async` pipe returns null before the Observable emits — combining it with `?.` or `??` is essential: `{{ (user$ | async)?.name ?? 'Loading...' }}`
- Angular's `@if` block (formerly `*ngIf`) is the structural alternative — it narrows the type inside the block so optional chaining is unnecessary: `@if (user) { {{ user.name }} }` — TypeScript knows `user` is non-null inside the block
- The nullish coalescing operator works in templates since Angular 9 — `{{ score ?? 0 }}` correctly handles a score of `0` where `{{ score || 0 }}` would not
- For complex nullable navigation, moving the null-guard logic into the component class and exposing a computed property or signal keeps templates clean

What a Weak Answer Misses: Weak answers demonstrate `?.` in templates but miss the `@if` narrowing pattern, which is often the cleaner alternative for complex nullable types because it eliminates the need for optional chaining entirely within the guarded block.

Question 3 — Senior

The Question: "You are reviewing a pull request for a component that uses optional chaining extensively throughout the template and the component class. The senior reviewer has left a comment saying 'this is a symptom, not a solution.' What do they mean, and how would you redesign the component?"

What a Strong Answer Covers:

- Pervasive `?.` throughout a component is a signal that the data model has not been designed to reflect when data is actually available
- The underlying problem: a single `User | undefined` type is being used across three distinct states — loading, loaded, and error — and the component has no way to distinguish them without checking every property individually
- The correct redesign: model the state explicitly as a discriminated union — type `ViewState = { status: 'loading' } | { status: 'loaded'; user: User } | { status: 'error'; message: string } — and expose it as a signal or observable`
- With a proper state model, the template uses `@if (state.status === 'loaded')` to narrow into a block where `user` is fully typed and non-null — optional chaining disappears because it is no longer needed

- Angular Signals reinforce this pattern: a computed signal that derives from a loading signal and a data signal produces a ViewState that the template consumes without any null navigation
- The senior reviewer's point: optional chaining is the right tool for genuinely optional data; when it appears on every property access it means the state model is incomplete, not that the data is inherently optional

What a Weak Answer Misses: Weak answers defend the optional chaining as correct defensive programming and miss the architectural insight that pervasive `?.` is a design smell indicating a missing state model.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Using `||` for Default Values on Numeric or Boolean Inputs

What it looks like:

```
@Component({
  selector: 'app-progress-bar',
  template: `<div [style.width.%]="displayValue">{{ displayValue }}%</div>`
})
export class ProgressBarComponent {
  @Input() value: number | undefined;
  @Input() showLabel: boolean | undefined;

  get displayValue(): number {
    return this.value || 100; // value of 0 renders as 100
  }

  get labelVisible(): boolean {
    return this.showLabel || true; // false always becomes true
  }
}
```

Why it seems fine at first: The component needs a default when the parent does not provide a value. `||` is the familiar JavaScript default-value pattern. It works correctly in every test case where value is undefined or null.

What breaks in production: A parent component binding `[value]="completedItems / totalItems"` produces 0 when no items are complete. The progress bar renders at 100% — the most misleading possible value. The `showLabel = false` case always renders the label. Both bugs are invisible in unit tests that only test the happy path.

The correct pattern:

```
export class ProgressBarComponent {
  @Input() value: number | undefined;
  @Input() showLabel: boolean | undefined;

  get displayValue(): number {
    return this.value ?? 100; // 0 is preserved; only undefined/null gets the default
  }

  get labelVisible(): boolean {
    return this.showLabel ?? true; // false is preserved; only undefined/null gets the default
  }
}
```

```
}
```

?? replaces || in every case where the input type includes legitimate falsy values. The rule is simple: if the type includes number, boolean, or empty string as valid values, use ??.

Anti-Pattern 2 — Optional Chaining Chains That Mask Missing Data Contracts

What it looks like:

```
@Component({
  selector: 'app-user-profile',
  template: `
<h1>{{ user?.profile?.personal?.displayName ?? 'Unknown' }}</h1>
<p>{{ user?.profile?.contact?.email ?? 'No email' }}</p>
<p>{{ user?.profile?.contact?.phone ?? 'No phone' }}</p>
<img [src]="user?.profile?.avatar?.url ?? defaultAvatar" />
<span>{{ user?.settings?.theme?.primary ?? '#000000' }}</span>
`
})
export class UserProfileComponent {
  @Input() user: User | undefined;
}
```

Why it seems fine at first: The template never throws. Every nullable access is defended. The component works regardless of how deeply nested the data arrives from the API. It feels robust.

What breaks in production: The template is impossible to read and impossible to test exhaustively — five separate chains, each with three to four levels, means sixteen possible undefined points. The component silently renders partial data when the API returns a malformed or incomplete User object.

The correct pattern:

```
// Define a view model with guaranteed non-null shape
interface UserProfileViewModel {
  displayName: string; email: string; phone: string;
  avatarUrl: string; primaryColour: string;
}

// Map in the service or a computed signal – null-guard once, at the boundary
function toViewModel(user: User | undefined): UserProfileViewModel {
  return {
    displayName: user?.profile?.personal?.displayName ?? 'Unknown',
    email: user?.profile?.contact?.email ?? 'No email',
    phone: user?.profile?.contact?.phone ?? 'No phone',
    avatarUrl: user?.profile?.avatar?.url ?? DEFAULT_AVATAR,
    primaryColour: user?.settings?.theme?.primary ?? '#000000'
  };
}

@Component({ selector: 'app-user-profile', template: `
<h1>{{ vm.displayName }}</h1>
<p>{{ vm.email }}</p>
<img [src]="vm.avatarUrl" />` })
export class UserProfileComponent {
  @Input() set user(value: User | undefined) { this.vm = toViewModel(value); }
```

```
vm: UserProfileViewModel = toViewModel(undefined);
}
```

Optional chaining happens once at the data boundary in a mapping function. The template receives a fully typed, non-null view model and needs no defensive operators at all.

Anti-Pattern 3 — Non-Null Assertion (!) Used Instead of Proper Null Guards

What it looks like:

```
@Component({
  selector: 'app-account',
  template: `<p>{{ user!.profile!.email }}</p>`
})
export class AccountComponent implements OnInit {
  user: User | undefined;

  ngOnInit(): void {
    this.userService.getUser().subscribe(u => {
      this.user = u;
    });
  }
}
```

Why it seems fine at first: The non-null assertion (!) tells TypeScript "trust me, this is not null." The developer knows the Observable will emit before the template renders — or thinks they do. TypeScript stops complaining.

What breaks in production: The template renders synchronously before the Observable emits. user is undefined on first render. ! suppresses the TypeScript error but cannot suppress the runtime TypeError.

The correct pattern:

```
@Component({
  selector: 'app-account',
  template: `
    @if (user) {
      <p>{{ user.profile.email }}</p>
    } @else {
      <p>Loading...</p>
    }
  `
})
export class AccountComponent {
  // Use the async pipe – no manual subscription, no lifecycle timing issues
  user$ = this.userService.getUser();
  constructor(private userService: UserService) {}
}
```

The async pipe handles subscription timing — the template receives null before the Observable emits and the @if block prevents any access to null properties. TypeScript narrows the type inside the @if block so no assertion is needed.

Section 16 — Mental Model Summary

Topic 19 — Optional Chaining & Nullish Coalescing | Mental Model

Core Concept in One Sentence:

Optional chaining (?.) and nullish coalescing (??) are precision null-safety tools — ?. short-circuits access on null or undefined to prevent `TypeError`s, and ?? provides defaults only for null or undefined while preserving every other falsy value including 0, false, and empty string.

The 6 Rules:

- 1. Use ?? instead of || whenever the type includes number, boolean, or string — || silently replaces 0, false, and "" with the default; ?? does not
- 2. ?. in templates is valid Angular syntax and composes naturally with the async pipe: `(user$ | async)?.name ?? 'Loading...'`
- 3. @if (value) in a template narrows the TypeScript type inside the block — inside a guarded @if block, optional chaining on that value is unnecessary
- 4. A chain longer than two levels (?. used three or more times in one expression) is a signal that a view model mapping function is missing
- 5. Never use the non-null assertion operator (!) as a substitute for a null guard — ! tells TypeScript to trust you and removes compile-time safety
- 6. Pervasive ?. throughout a component is a design smell indicating a missing state model — model loading, loaded, and error as a discriminated union

The #1 Angular Rule: Never use || to provide a default for a numeric or boolean @Input() — use ?? instead, because a parent binding of [value]="0" or [enabled]="false" is valid data that || will silently replace with your default.

Quick Reference:

Operator	Returns right side when	Preserves 0, false, ""	Angular Use Case
	Left side is falsy	No	String-only defaults where falsy = absent
??	Left side is null/undefined only	Yes	@Input() defaults, async pipe results
?.	Left side is null/undefined	Yes	Nullable object navigation, async pipe access
!	Never — suppresses check	Yes	Avoid in production — use a real null guard
@if (val)	—	No (falsy blocks rendering)	Template null narrowing — cleanest option

The One Thing to Remember:

The difference between || and ?? is the difference between "this value is absent" and "this value is falsy" — in Angular where 0 progress, false toggles, and empty strings are all legitimate data, using || for defaults is a bug waiting to happen on the first edge case a QA engineer or real user discovers.

TOPIC 25 — Component Lifecycle Hooks

Angular | Intermediate | Very High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 17 — TypeScript Decorators (@Component is the decorator that registers a class as an Angular component and makes lifecycle hooks meaningful; without understanding what @Component does, lifecycle hooks are just magic method names)
- Topic 15 — Access Modifiers (lifecycle hook methods must be public — Angular's runtime calls them from outside the class; understanding why requires access modifier knowledge)
- Topic 19 — Optional Chaining & Nullish Coalescing (@Input() properties are undefined before ngOnChanges fires; safe access patterns are essential inside early lifecycle methods)
- Topic 8 — Promises & async/await (ngOnInit is the most common place for async data fetching; understanding async patterns is a prerequisite for using the lifecycle correctly)
- Topic 18 — Error Handling (subscriptions started in lifecycle hooks must be cleaned up in ngOnDestroy; error handling and lifecycle management are inseparable in production components)

Unlocks:

- Topic 26 — @Input() & @Output() in Depth (ngOnChanges fires on every @Input() change and receives a SimpleChanges object; you cannot use @Input() at production depth without understanding the lifecycle hook that responds to it)

Production Readiness Assessment:

After studying lifecycle hooks alone you can use ngOnInit, ngOnDestroy, and ngOnChanges correctly in straightforward components immediately — this topic is production-ready in isolation for those three hooks, but ngAfterViewInit, ngAfterContentInit, and the nuances of OnPush interaction require combining this with Topic 26 (@Input/@Output) and a working understanding of Angular's change detection.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "What is the difference between a component's constructor and ngOnInit, and what logic belongs in each?"

What a Strong Answer Covers:

- The constructor runs before Angular has processed the component — @Input() properties are not yet bound, the view is not yet created, and Angular's DI system is the only thing that has run
- ngOnInit runs after Angular has set all @Input() bindings for the first time — it is the correct place for initialisation logic that depends on input values
- Constructor is for dependency injection and simple property initialisation only — never for HTTP calls, never for reading @Input() values, never for DOM access
- ngOnInit is for data fetching, subscription setup, and any initialisation that requires @Input() values to be available

- Practical rule: if the logic would fail without Angular having processed the component first, it belongs in `ngOnInit`; if it only needs the constructor parameters, it belongs in the constructor

What a Weak Answer Misses: Weak answers say "constructor is for DI, `ngOnInit` is for everything else" without explaining why — specifically that `@Input()` values are not available in the constructor and that putting HTTP calls in the constructor produces code that is difficult to test because the call fires before any test setup can intercept it.

Question 2 — Mid

The Question: "How does `ngOnChanges` work, when does it fire, and how would you use it to react to a specific `@Input()` changing without running expensive logic when other inputs change?"

What a Strong Answer Covers:

- `ngOnChanges` fires before `ngOnInit` on first render and then again every time any `@Input()` binding changes — it receives a `SimpleChanges` object keyed by input property name
- Each entry in `SimpleChanges` carries `previousValue`, `currentValue`, and `firstChange` — `firstChange` is true only on the initial binding, not on subsequent updates
- To react to a specific input: check `changes['propertyName']` before running logic — if the key is absent the input did not change this cycle
- To avoid running expensive logic on first change when you only care about updates: guard with `if (!changes['propertyName'].firstChange)`
- For `OnPush` components, `ngOnChanges` still fires on reference changes to inputs — it is one of the triggers that marks the component for check even in `OnPush` mode
- Alternative pattern for simple cases: use an `@Input()` setter, which fires only for that specific property and provides the new value directly without parsing `SimpleChanges`

What a Weak Answer Misses: Weak answers describe `ngOnChanges` at a surface level but miss the `firstChange` guard pattern — which is the most common real-world use case — and do not mention the `@Input()` setter as a cleaner alternative for single-property reactions.

Question 3 — Senior

The Question: "You are building a high-performance dashboard with thirty `OnPush` components. Several are experiencing stale views where the data has changed but the template has not updated. Walk me through how you would diagnose and fix this without removing `OnPush` from any component."

What a Strong Answer Covers:

- `OnPush` components only re-render when: an `@Input()` reference changes, an async pipe's `Observable` emits, an event originates from within the component, or `ChangeDetectorRef.markForCheck()` is called — anything outside these four triggers produces a stale view
- Diagnosis step 1: identify whether the stale component is receiving data through an `@Input()` or through a service subscription — the fix is different for each
- For `@Input()` stale views: the parent is mutating an object in place instead of producing a new reference — fix by spreading into a new object before passing down
- For service subscription stale views: the subscription is updating a property but not triggering change detection — fix by using the async pipe with an `Observable` rather than

manual subscribe(), or by calling this.cdr.markForCheck() after updating state

- For computed state derived from multiple signals: use Angular Signals — a computed() signal automatically marks dependant OnPush components for check when any source signal changes, eliminating the need to manually call markForCheck()
- The architectural fix: prefer exposing state as Observables or Signals from services rather than raw values — the async pipe and signal binding both integrate with OnPush change detection automatically

What a Weak Answer Misses: Weak answers suggest removing OnPush or calling detectChanges() everywhere — detectChanges() runs a full CD cycle on the component subtree and is a performance regression that defeats the purpose of OnPush; the correct tools are markForCheck(), the async pipe, and Signals, each used in the appropriate context.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Starting Subscriptions in ngOnInit Without Cleaning Up in ngOnDestroy

What it looks like:

```
@Component({
  selector: 'app-notifications',
  template: `<div *ngFor="let n of notifications">{{ n.message }}</div>`
})
export class NotificationsComponent implements OnInit {
  notifications: Notification[] = [];

  constructor(private notificationService: NotificationService) {}

  ngOnInit(): void {
    // Subscription started – never cleaned up
    this.notificationService.notifications$.subscribe(notifications => {
      this.notifications = notifications;
    });
  }
}
```

Why it seems fine at first: The component works correctly while it is active. Notifications update in real time. There are no visible errors. In a simple application that never destroys this component the leak never manifests.

What breaks in production: Every time the component is created — navigating to a route, opening a modal, toggling a tab — a new subscription is added to the Observable. When the component is destroyed the subscription is not removed. In a long-running single-page application with frequent navigation, dozens of dead subscriptions accumulate, each holding a reference to the destroyed component and preventing garbage collection.

The correct pattern:

```
// First choice: async pipe manages subscription lifetime automatically
@Component({
  selector: 'app-notifications',
  template: `<div *ngFor="let n of (notifications$ | async)">{{ n.message }}</div>`
})
export class NotificationsComponent {
  notifications$ = this.notificationService.notifications$;
  constructor(private notificationService: NotificationService) {}
}
```

```

}

// If manual subscription is genuinely needed:
export class NotificationsComponent implements OnInit {
  notifications: Notification[] = [];

  constructor(
    private notificationService: NotificationService,
    private destroyRef: DestroyRef
  ) {}

  ngOnInit(): void {
    this.notificationService.notifications$
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(notifications => {
        this.notifications = notifications;
      });
  }
}

```

The async pipe is the first choice — it subscribes on init, unsubscribes on destroy, and triggers OnPush change detection automatically. When a manual subscription is unavoidable, `takeUntilDestroyed()` with `DestroyRef` is the Angular 16+ idiomatic cleanup pattern.

Anti-Pattern 2 — Accessing `@ViewChild()` References in `ngOnInit`

What it looks like:

```

@Component({
  selector: 'app-video-player',
  template: `<video #player [src]="videoUrl"></video>`
})
export class VideoPlayerComponent implements OnInit {
  @ViewChild('player') playerRef!: ElementRef<HTMLVideoElement>;
  @Input() videoUrl!: string;
  @Input() autoPlay = false;

  ngOnInit(): void {
    if (this.autoPlay) {
      // playerRef is undefined here – the view has not been initialised
      this.playerRef.nativeElement.play();
    }
  }
}

```

Why it seems fine at first: `ngOnInit` feels like the right initialisation hook. The `@ViewChild()` is declared and typed. The non-null assertion suppresses TypeScript's warning. In some rendering scenarios with simple synchronous templates it may intermittently work in development.

What breaks in production: `@ViewChild()` references are populated after the view is created, which happens after `ngOnInit` completes. `this.playerRef` is undefined in `ngOnInit` — the non-null assertion tells TypeScript to trust the developer but the runtime does not honour that trust. In production, calling `.play()` on undefined throws a `TypeError`.

The correct pattern:

```

@Component({
  selector: 'app-video-player',
  template: `<video #player [src]="videoUrl"></video>`
})
export class VideoPlayerComponent implements AfterViewInit {
  @ViewChild('player') playerRef!: ElementRef<HTMLVideoElement>;
  @Input() videoUrl!: string;
  @Input() autoPlay = false;

  ngAfterViewInit(): void {
    // playerRef is guaranteed to be available here
    if (this.autoPlay) {
      this.playerRef.nativeElement.play();
    }
  }
}

```

`@ViewChild()` references are read in `ngAfterViewInit` — this is the earliest lifecycle hook at which the view DOM exists and the reference is populated.

Anti-Pattern 3 — Triggering Expensive Logic in `ngOnChanges` Without Guarding Against Irrelevant Changes

What it looks like:

```

@Component({ selector: 'app-data-grid', template: `...` })
export class DataGridComponent implements OnChanges {
  @Input() rows: Row[] = [];
  @Input() pageSize = 10;
  @Input() sortColumn = 'name';
  @Input() theme = 'light';

  ngOnChanges(changes: SimpleChanges): void {
    // Runs on EVERY input change – including theme changing from 'light' to 'dark'
    this.processAndSortRows(); // expensive – O(n log n)
    this.rebuildVirtualScroller(); // very expensive – recreates DOM nodes
  }
}

```

Why it seems fine at first: `ngOnChanges` is the correct hook for reacting to input changes. Running the full recalculation ensures the view is always consistent. In development with small datasets the performance cost is invisible.

What breaks in production: Every time the parent changes any input — including cosmetic inputs like theme — the full expensive pipeline runs. With 10,000 rows this freezes the UI. The component is doing expensive data work in response to a CSS class change.

The correct pattern:

```

ngOnChanges(changes: SimpleChanges): void {
  // Only re-sort when data inputs change – not when theme changes
  const dataChanged = changes['rows'] || changes['pageSize'] || changes['sortColumn'];

  if (dataChanged) {
    this.processAndSortRows();
    this.rebuildVirtualScroller();
  }
}

```

```
// Theme change is cheap – handle it separately
if (changes['theme']) {
  this.applyTheme(this.theme);
}
}
```

Each expensive operation is guarded by checking whether the specific inputs that justify it have actually changed. Cheap operations like theme application are separated from expensive data operations.

Section 16 — Mental Model Summary

Topic 25 — Component Lifecycle Hooks | Mental Model

Core Concept in One Sentence:

Angular's lifecycle hooks are a sequence of guaranteed checkpoints at which your component can safely interact with its inputs, its view, its content, and the change detection system — and knowing exactly which resources are available at each checkpoint is what separates correct lifecycle usage from intermittent bugs.

The 6 Rules:

- 1. Constructor is for dependency injection only — `@Input()` values, the view, and projected content are all unavailable
- 2. `ngOnInit` is for data fetching and input-dependent initialisation — `@Input()` values are available, the view is not
- 3. `ngOnChanges` fires before `ngOnInit` on first render and on every subsequent `@Input()` reference change — always check `changes['propertyName']` and `firstChange` before running expensive logic
- 4. `ngAfterViewInit` is the earliest point at which `@ViewChild()` and `@ViewChildren()` references are available — never access them before this hook
- 5. `ngOnDestroy` is mandatory whenever the component creates subscriptions, timers, or event listeners — prefer `takeUntilDestroyed()` and the async pipe over manual cleanup
- 6. `OnPush` components re-render only on `@Input()` reference change, async pipe emission, internal events, or `markForCheck()` — stale views in `OnPush` are always caused by one of these four triggers being absent

The #1 Angular Rule: Never access a `@ViewChild()` reference in `ngOnInit` — it is undefined until `ngAfterViewInit` fires, and the non-null assertion operator will not save you from the runtime `TypeError`.

Quick Reference:

Hook	<code>@Input()</code> Available	View Available	Content Available	Common Use
constructor	No	No	No	DI only
<code>ngOnChanges</code>	Yes	No	No	React to input changes
<code>ngOnInit</code>	Yes	No	No	Data fetching, setup
<code>ngAfterContentInit</code>	Yes	No	Yes	<code>@ContentChild()</code> access
<code>ngAfterViewInit</code>	Yes	Yes	Yes	<code>@ViewChild()</code> access, DOM

ngOnDestroy	Yes	Yes	Yes	Cleanup, unsubscribe
-------------	-----	-----	-----	----------------------

The One Thing to Remember:

Every subscription, timer, and global event listener started in a lifecycle hook is a contract — the component promised Angular it would clean up after itself, and `ngOnDestroy` is where that promise is kept; a component that starts things without stopping them is a memory leak waiting to accumulate across a thousand navigations.

TOPIC 26 — @Input() & @Output() in Depth

Angular | Intermediate-Advanced | Very High Relevance

Section 13 — Study Plan Checkpoint

Dependencies:

- Topic 15 — Access Modifiers (@Input() and @Output() properties must be public — this is a hard Angular compiler requirement; the reasoning is grounded in access modifier knowledge)
- Topic 17 — TypeScript Decorators (@Input() and @Output() are property decorators; understanding what decorators do at compile time explains how Angular reads these annotations to wire the component API)
- Topic 25 — Component Lifecycle Hooks (ngOnChanges fires on every @Input() change and is the primary reactive hook for input-driven logic; this topic is the direct continuation of that hook's story)
- Topic 16 — TypeScript Generics (EventEmitter and the new signal-based input() and output() functions are generic; reading and writing their type signatures requires generics knowledge)
- Topic 19 — Optional Chaining & Nullish Coalescing (@Input() properties are undefined before the parent binds them; safe access patterns are essential for robust component APIs)

Unlocks:

This is the final topic in the current series. Mastering @Input() and @Output() in depth consolidates every prior topic into the component boundary — the most important architectural surface in any Angular application. In a broader Angular learning path this topic directly enables: Content Projection (ng-content), Dynamic Components, Angular CDK, and any serious work with component libraries or design systems.

Production Readiness Assessment:

After studying @Input() and @Output() in depth you are production-ready at the component boundary immediately — this topic synthesises access modifiers, decorators, generics, lifecycle hooks, and null safety into the single most common pattern in Angular development, and a developer who is solid on all eight topics in this series can design, build, and review component APIs with genuine confidence in a real production codebase.

Section 14 — Common Interview Questions

Question 1 — Junior

The Question: "What is the purpose of @Input() and @Output() in Angular, and how does data flow between a parent and child component using them?"

What a Strong Answer Covers:

- @Input() allows a parent component to pass data down to a child — it is the mechanism for one-way data binding from parent to child
- @Output() allows a child component to emit events up to a parent — it is the mechanism for child-to-parent communication
- @Output() properties are always typed EventEmitter — the generic T defines the shape of the event payload the parent receives

- Data flows in one direction: down through `@Input()`, up through `@Output()` — this is Angular's enforcement of the unidirectional data flow principle
- The parent binds to an `@Input()` with square bracket syntax `[inputName]="value"` and listens to an `@Output()` with parenthesis syntax `(outputName)="handler($event)"`
- Two-way binding with `[(ngModel)]` or `[(value)]` is syntactic sugar for an `@Input()` named `value` and an `@Output()` named `valueChange` — Angular expands it into both bindings

What a Weak Answer Misses: Weak answers describe `@Input()` and `@Output()` in isolation without explaining the directional data flow principle, and rarely mention that two-way binding is just syntactic sugar for a matched `@Input()/@Output()` pair — which is the most common follow-up question in Angular interviews.

Question 2 — Mid

The Question: "What is the difference between using an `@Input()` setter and using `ngOnChanges` to react to input changes, and when would you choose each?"

What a Strong Answer Covers:

- An `@Input()` setter fires only when that specific property changes, receives the new value directly as its parameter, and is colocated with the property declaration — making intent immediately clear
- `ngOnChanges` fires when any `@Input()` changes, receives a `SimpleChanges` map for all changed inputs, and must explicitly check which input changed before acting
- Setter is the right choice when reacting to a single input — validation, transformation, derived state computation; the code is simpler and the property and its reaction logic are in one place
- `ngOnChanges` is the right choice when the reaction depends on the relationship between multiple inputs — for example, recalculating a derived value only when both `rows` and `sortColumn` have changed in the same cycle
- Setter anti-pattern: running expensive side effects in a setter that fires frequently — the setter runs synchronously on every binding cycle; heavy computation should be deferred or memoised
- Angular 17+ signal inputs (`input()`) replace both patterns for most cases — a `computed()` signal derived from an input signal reacts automatically to changes without a setter or `ngOnChanges`

What a Weak Answer Misses: Weak answers treat setters and `ngOnChanges` as equivalent alternatives and pick one based on personal preference, missing that the correct choice is determined by whether the reaction involves one input or the relationship between multiple inputs.

Question 3 — Senior

The Question: "You are designing a reusable data table component that will be used by twelve feature teams across a large Angular application. It needs to support custom column definitions, server-side pagination, row selection, and an action menu. How would you design the `@Input()` and `@Output()` API surface, and what principles guide your decisions?"

What a Strong Answer Covers:

- Principle 1 — minimal surface: expose only the inputs the parent needs to control; derive everything else internally; every additional `@Input()` is a commitment to maintain backward compatibility
- Column definitions as a typed interface input: `@Input() columns: ColumnDef[]` where `ColumnDef` carries field, header, sortable, and an optional `cellTemplate: TemplateRef`
- Pagination as a controlled input/output pair: `@Input() pageConfig: PageConfig` and `@Output() pageChange = new EventEmitter()` — the parent owns pagination state, the table requests changes; this enables server-side pagination without the table holding state it cannot own
- Row selection output: `@Output() selectionChange = new EventEmitter()` — typed to the generic row type so the parent receives strongly-typed selected rows, not `any[]`
- Action menu: expose as `ng-content` projection rather than an `@Input()` — allows feature teams to pass arbitrary action components without the table needing to know about them
- Principle 2 — the table should be a controlled component for state it cannot decide alone (pagination, selection) and uncontrolled for state it owns completely (sort direction within a page, column resizing)
- Principle 3 — use Angular's new signal-based `input()` for the Angular 17+ API; it integrates with `OnPush` change detection automatically and removes the need for `ngOnChanges` in the table's internal logic

What a Weak Answer Misses: Weak answers design a flat list of `@Input()` properties for every feature without applying any organising principle, miss the controlled vs uncontrolled component distinction for pagination and selection state, and do not consider `ng-content` projection as an alternative to action-related inputs.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Mutating `@Input()` Arrays or Objects Directly in the Child Component

What it looks like:

```
@Component({
  selector: 'app-task-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
<div *ngFor="let task of tasks">
  {{ task.title }}
  <button (click)="complete(task)">Complete</button>
</div>
`
})
export class TaskListComponent {
  @Input() tasks: Task[] = [];

  complete(task: Task): void {
    // Directly mutating the input array – parent owns this reference
    task.completed = true;
    this.tasks.splice(this.tasks.indexOf(task), 1);
  }
}
```

Why it seems fine at first: The mutation appears to work — the completed task disappears from the list. In Default change detection mode Angular catches the change on the next cycle.

The simplicity is appealing; there is no extra ceremony.

What breaks in production: With `OnPush` the view does not re-render after `splice()` because the array reference never changed — Angular sees the same reference and skips the check. The parent's state is silently corrupted because the child mutated data it does not own.

The correct pattern:

```
export class TaskListComponent {
  @Input() tasks: Task[] = [];
  @Output() taskCompleted = new EventEmitter<Task>();

  complete(task: Task): void {
    // Emit up — parent decides how to update state
    this.taskCompleted.emit({ ...task, completed: true });
  }
}

// Parent handles state update — produces a new reference
onTaskCompleted(completedTask: Task): void {
  this.tasks = this.tasks.filter(t => t.id !== completedTask.id);
}
```

The child emits a new object upward. The parent produces a new array reference. `OnPush` fires correctly because the reference changed. State ownership is clear and traceable.

Anti-Pattern 2 — Overloading a Single `@Input()` with Multiple Responsibilities

What it looks like:

```
// A config object that tries to do everything
interface ButtonConfig {
  label: string; type: 'primary' | 'secondary' | 'danger';
  size: 'sm' | 'md' | 'lg'; disabled: boolean; loading: boolean;
  icon: string; iconPosition: 'left' | 'right';
  tooltipText: string; analyticsId: string;
}

@Component({ selector: 'app-button', template: `...` })
export class ButtonComponent {
  @Input() config!: ButtonConfig;
}

// Parent usage — verbose and opaque
<app-button [config]="{
  label: 'Save', type: 'primary', size: 'md', disabled: false,
  loading: isSaving, icon: 'save', iconPosition: 'left',
  tooltipText: 'Save changes', analyticsId: 'save-btn'
}" />
```

Why it seems fine at first: A single `@Input()` feels clean — one property to bind, one object to pass. It avoids a long list of individual property bindings in the template. Config objects are a familiar pattern from JavaScript libraries.

What breaks in production: Every time any property changes, the entire config object reference must change for `OnPush` to detect it — or the parent must remember to spread the config on every partial update. Partial updates are error-prone: `this.config.disabled = true` silently

breaks OnPush. Adding a new required property to ButtonConfig is a breaking change for every team using the component.

The correct pattern:

```
@Component({ selector: 'app-button', template: `...` })
export class ButtonComponent {
  // Individual inputs – each independently bindable and OnPush-compatible
  @Input() label = '';
  @Input() variant: 'primary' | 'secondary' | 'danger' = 'primary';
  @Input() size: 'sm' | 'md' | 'lg' = 'md';
  @Input() disabled = false;
  @Input() loading = false;
  @Input() icon?: string;
  @Input() iconPosition: 'left' | 'right' = 'left';
  @Input() tooltip?: string;
  @Input() analyticsId?: string;
}

// Parent usage – readable, partial binding supported natively
<app-button label="Save" variant="primary" [loading]="isSaving" icon="save" />
```

Individual `@Input()` properties allow partial binding — the parent only specifies what it needs to override. Each property change is detected independently by OnPush. Adding a new optional input is a non-breaking change.

Anti-Pattern 3 — Using `@Output()` with EventEmitter for State That Belongs in a Shared Service

What it looks like:

```
// Three levels of components – event drilling upward
@Component({ selector: 'app-row-action', template: `...` })
export class RowActionComponent {
  @Output() actionTriggered = new EventEmitter<Action>();
  onDelete(): void { this.actionTriggered.emit({ type: 'delete', id: this.rowId }); }
}

@Component({ selector: 'app-table-row', template: `...` })
export class TableRowComponent {
  @Output() actionTriggered = new EventEmitter<Action>();
  onAction(action: Action): void { this.actionTriggered.emit(action); } // just passing through
}

@Component({ selector: 'app-data-table', template: `...` })
export class DataTableComponent {
  @Output() actionTriggered = new EventEmitter<Action>(); // still passing through
  onAction(action: Action): void { this.actionTriggered.emit(action); }
}

// Parent finally handles it – four levels up from origin
```

Why it seems fine at first: `@Output()` is the correct Angular pattern for child-to-parent communication. Each component passes the event up. The architecture looks clean on a component diagram where each level has a clear `@Output()`.

What breaks in production: TableRowComponent and DataTableComponent are pure pass-throughs — they add @Output() declarations and event handlers that do nothing except relay the event upward. Every intermediate component now has an API surface that must be maintained, documented, and tested even though it has no opinion about the event. Adding a new action type requires touching every intermediate component.

The correct pattern:

```
// Action events go directly to a service – no drilling
@Injectable({ providedIn: 'root' })
export class TableActionService {
  private readonly actionSubject$ = new Subject<Action>();
  public readonly action$ = this.actionSubject$.asObservable();

  dispatch(action: Action): void {
    this.actionSubject$.next(action);
  }
}

// Deep child – injects service directly, no @Output() chain needed
@Component({ selector: 'app-row-action', template: `...` })
export class RowActionComponent {
  constructor(private tableAction: TableActionService) {}
  onDelete(): void { this.tableAction.dispatch({ type: 'delete', id: this.rowId }); }
}

// Feature component – subscribes to service directly
export class FeatureComponent implements OnInit {
  constructor(private tableAction: TableActionService, private destroyRef: DestroyRef) {}

  ngOnInit(): void {
    this.tableAction.action$
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(action => this.featureService.processAction(action));
  }
}
```

The service is the communication channel. Intermediate components have no @Output() declarations and no event-handling boilerplate. Adding a new action type touches only the origin component and the feature handler. @Output() is reserved for direct parent-child communication where the parent genuinely owns the response.

Section 16 — Mental Model Summary

Topic 26 — @Input() & @Output() in Depth | Mental Model

Core Concept in One Sentence:

@Input() and @Output() define a component's public API contract — @Input() is the interface through which a parent passes data down, @Output() is the interface through which a child signals intent upward, and the discipline of keeping that boundary clean is what makes Angular components reusable, testable, and maintainable at scale.

The 6 Rules:

- 1. Never mutate an `@Input()` — always emit a new value upward through `@Output()` and let the parent decide how to update state
- 2. `@Input()` and `@Output()` must always be public — the Angular compiler requires it and strict template type checking enforces it
- 3. Use an `@Input()` setter for single-input reactions; use `ngOnChanges` when the reaction depends on the relationship between multiple inputs simultaneously
- 4. Prefer individual `@Input()` properties over a single config object `@Input()` — individual properties are independently bindable, `OnPush`-compatible, and partially overridable without spreading
- 5. Use `@Output()` only for direct parent-child communication — when events need to travel more than one level up, a shared service is the correct channel
- 6. For Angular 17+, prefer signal-based `input()` and `output()` over decorator-based `@Input()` and `@Output()` — they integrate with `OnPush` automatically and eliminate the need for `ngOnChanges` in most cases

The #1 Angular Rule: A child component that mutates its `@Input()` is a component that has taken ownership of state it does not own — this silently corrupts `OnPush` change detection, makes state untraceably shared, and is the single most common cause of data consistency bugs at the component boundary.

Quick Reference:

Scenario	Correct Pattern	Wrong Pattern
Pass data parent to child	<code>@Input()</code> property	Service with shared mutable state
Child signals intent to parent	<code>@Output()</code> <code>EventEmitter<T></code>	Direct parent method call
React to one input changing	<code>@Input()</code> setter	<code>ngOnChanges</code> checking one key
React to multiple inputs together	<code>ngOnChanges</code> with guards	Multiple setters with shared flag
Events crossing multiple levels	Shared service <code>Subject</code>	<code>@Output()</code> event drilling
Two-way binding	<code>@Input()</code> value + <code>@Output()</code> <code>valueChanges</code>	Direct property mutation
Modern Angular 17+ inputs	<code>input<T>()</code> signal function	<code>@Input()</code> decorator (legacy)
Default for optional input	<code>@Input()</code> <code>prop = defaultValue</code>	Nullish coalescing in template

The One Thing to Remember:

The component boundary is a contract — every `@Input()` is a promise that the parent can control this aspect of the child's behaviour, and every `@Output()` is a promise that the child will signal this event without acting on it unilaterally; a component that mutates its inputs or acts on events without emitting them has broken its contract with every team that depends on it.
